

Chapter 1

C# Fundamentals — Syntax, Types, Methods, Arrays

ENEL712 Embedded Systems Design — Lecture 19

Summary

Core C# language fundamentals: syntax rules, data types and conversion, methods and parameter passing (ref/out/params), operators, arrays (with stack/heap semantics), and exception handling.

Topics Covered

- C# fundamentals and syntax
- Variable naming
- Data types and conversion
- Methods and parameters
- Operators and mathematical functions
- Arrays
- Exception handling

1.1 C# Fundamentals and Syntax

- Statements end with `;`. Whitespace doesn't terminate; statements can span multiple lines.
- Identifiers are case-sensitive and must start with a letter or underscore.
- Comments: `//` single-line; `/* ... */` multi-line.
- Reserved keywords: `abstract`, `class`, `if`, `namespace`, `public`, `static`, `void`, etc.

1.2 Variable Naming

- Start with a lowercase letter; avoid leading underscores.
- Avoid names differing only in case (`var` vs `Var`).

- Use camelCase for multi-word variables (myVar, fileID).

1.3 Data Types and Conversion

Type	Kind	Size (bits)	Range / precision
byte	Whole numbers	8	0 to 255
int	Whole numbers	32	-2^{31} to $2^{31}-1$
long	Whole numbers	64	-2^{63} to $2^{63}-1$
float	Floating point	32	—
double	Floating point	64	—
decimal	Monetary	128	28 significant figures
char	Single character	16	0 to $2^{16}-1$
string	Sequence of chars	16/char	—
bool	Boolean	8	true / false

- Unlike C++, using an unassigned local variable is a compile-time error.
- Implicit cast when no information is lost (float $\rightarrow$ double); explicit cast otherwise.
- Convert.ToInt32() (returns 0 for nulls) vs Int32.Parse() (throws on null).
- Numbers to strings: .ToString().

1.4 Methods and Parameters

- C# methods pass by value by default — receiver gets a copy.
- ref: pass by reference; variable must be initialised before passing.
- out: pass by reference but allows unassigned variables; must be initialised inside the function.
- params: variable number of arguments received as a single array (distinct from overloading).

1.5 Operators and Mathematical Functions

1.5.1 Operator precedence (high $\rightarrow$ low)

- Primary: x.y, f(x), a[x], x++, new
- Unary: +, −, !, ~, ++x, −−x, (T)x
- Multiplicative: *, /, %
- Additive: +, −
- Relational: <, >, <=, >=, is, as
- Logical: &, ^, |
- Conditional: &&, ||, ?:
- Assignment: =, *=, /=, %=, +=, −=, ...

1.5.2 Math, randoms, dates

- System.Math: Abs, Sin, Sqrt, Pow...
- Random class: pseudo-random ints/doubles.
- DateTime: dates/times/leap years (e.g., DateTime.Now, AddDays).

1.6 Arrays

- Ordered sequence of same-type elements stored contiguously.
- Declare with brackets after the type (int[] pins); allocate with new.
- Reference lives on the stack; element memory lives on the heap. Numeric elements default to 0.
- Iterate with for + .Length or with foreach.
- Assignment copies the reference; use pins.CopyTo(copy, 0) to duplicate contents.
- Multidimensional rectangular arrays (int[,]) and jagged arrays (arrays of arrays).

1.7 Exception Handling

- try block contains code that may throw.
- catch block handles specific exceptions (e.g., FormatException) or all via catch(Exception ex).
- ex.Message provides error details for logging or user feedback.

1.8 Use Cases & Applications

- ref params: useful for in-place updates of a struct that would otherwise be copied (e.g., a large GameState).
- out params: classic Try-pattern, e.g., int.TryParse(s, out int n) — succeeds without throwing.
- params arrays: variadic methods like Console.WriteLine(format, params object[] args).
- Type conversions: Int32.Parse vs Convert.ToInt32 — Parse is strict, Convert is more lenient and may swallow nulls.
- Jagged arrays: triangular tables (e.g., Pascal's triangle, lower-triangular matrices) where each row has a different length.

1.9 Worked Example: TryParse Pattern

```
if (int.TryParse(textBox.Text, out int hours))
    label.Text = $"You entered {hours} hours.";
else
    label.Text = "Please enter an integer.";
```

TryParse is preferred over Parse + try/catch in hot paths — exceptions are expensive in .NET (kilobytes of stack walk per throw).

1.10 Worked Example: Copy-by-Reference Pitfall

```
int[] a = { 1, 2, 3 };
int[] b = a;           // copies the reference, not the data
b[0] = 99;
Console.WriteLine(a[0]); // prints 99

int[] c = (int[])a.Clone(); // or new int[] { ... }
c[0] = -1;
Console.WriteLine(a[0]); // prints 99 (independent now)
```

1.11 Operator Precedence Cheat Sheet

Group (high → low)	Operators
Primary	x.y, f(x), a[x], x++, new
Unary	+, -, !, ~, ++x, -x, (T)x
Multiplicative	*, /, %
Additive	+, -
Relational	<, >, <=, >=, is, as
Logical	&, ^,
Conditional	&&, , ?:
Assignment	=, *=, /=, %=, +=, -=, ...

Key Definitions

ref Method parameter modifier passing by reference; arg must be initialised.

out

Method parameter modifier passing by reference; arg may be unassigned but must be set inside.

params

Modifier letting a method accept a variable number of args as a single array.

Stack

LIFO memory region for locals and reference variables.

Heap

Memory region for dynamic allocation (e.g., array contents and class instances).

Dynamic Memory Allocation

Allocating memory at runtime, typically via new.

Jagged Array

Array of arrays where inner arrays may have different lengths.

Operator Precedence

Rules for the order in which operators evaluate.

System.Math

Static class with mathematical constants and functions.

Exception Handling

try/catch mechanism for handling runtime errors gracefully.

Boxing

Wrapping a value-type instance into a heap-allocated object; happens when you assign an int to an object variable.

Unboxing

Extracting the value from a boxed object back into its value type; requires a runtime cast.

params Modifier

Lets a method accept a variable number of arguments collected into an array.

TryParse Pattern

Static method returning bool that signals success and writes the parsed value to an out parameter.

Sources

- Lecture 19.pdf